Project Title

Tim Fan (SID: 550790019)
hourlilies
https://github.com/hourlilies/elec5305-project-550790019

I. Project Overview

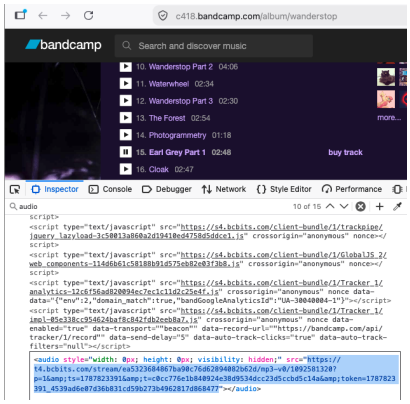
This project aims to develop a synthesizer, sampler, and music sequencer for any device capable of running a modern web browser supporting WebAudio, using the emscripten compiler toolchain that turns C++ DSP code into WebAssembly. The goal is to make music production more accessible, because pretty much any computer, whether that be a smartphone, laptop, or even a single-board computer, can run a modern web browser that supports WebAudio. This is important because anyone should be able to play with and make music without needing to install and learn a comprehensive digital audio workstation, designed for professional musicians and not the general public.

Though the advent of generative AI in music creation has lowered the barrier of entry tremendously, empowering authors to make and tune music to their taste with just a few text prompts and a bit of luck, it has come at the cost of abstracting away the underlying signal processing and audio intuitions that makes it all work. This project aims to expose all of that, the wires, knobs, and switches of music making, with the hopes of allowing the prospective musician to see and poke at the machine that makes modern music tick, whose human invention and decades of production is what allows powerful GenAI tools like Suno or Udio to even exist.

II. Background and Motivation

Despite its relatively recent history, with the first publication of a working draft in only 2011[1], the WebAudio API has seen extensive development over the course of a decade, eventually receiving an endorsement from the W3C in 2021 as a Recommendation[2]. Notably, its primary `AudioContext` interface has been fully supported by most major web browsers since at least 2015[3], with the only exception being Safari which implemented support in 2021[4].

Of course, audio has existed for a much longer time than that on the internet. As Boris Smus describes in their brief history of audio on the web [1], initially that was through the Flash external plugin for browsers, and then later through the HTML5 `<audio>` element which was instead natively supported by the web browser itself. In fact, it's this `<audio>` element that allows music streaming websites like Bandcamp to preview an artist's music through the web browser (see Fig. 1).




```
Input ... from '1092581320.mp3' Duration:
  00:02:48.00, ..., bitrate: 236 kb/s
```

Fig. 1. Bandcamp allows artists to offer their listeners a compressed and lower bitrate MP3 preview of their tracks, played in the browser through the HTML5 `<audio>` element. The text above is the result of `ffprobe 1092581320.mp3`

But one thing Flash's plugin did for web audio that HTML5's `<audio>` element doesn't do, is allow for interactive audio. For example, a synthesizer/sampler/sequencer like the one this project will aim to build, would require a predictable amount of latency between a person pressing a key on their keyboard and the corresponding sound being played through their speakers, something that the `<audio>` element just wasn't designed for.

That is the kind of problem the WebAudio API attempts to solve, and why it has been possible for synthesizers and other real time instruments to be built for the web. For example stevengoldberg's emulation of the Juno-106 synthesizer[5] built for the web in 2015, to ryukau's collection of synthesizers[6] build for the web which use the aforementioned `AudioContext` interface as the backbone of its audio framework (see Fig. 2).

III. Proposed Methodology

methodology.tex

IV. Expected Outcomes

outcomes.tex

V. Timeline

timeline.tex

[1] https://www.w3.org/TR/2011/WD-webaudio-20111215/
[2] https://www.w3.org/TR/webaudio-1.0/
[3] https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API#browser_compatibility
[4] https://developer.apple.com/documentation/safari-release-notes/safari-14_1-release-notes#WebKit-Framework
[5] https://github.com/stevengoldberg/juno106
[6] https://github.com/ryukau/UhhyouWebSynthesizers

```
// Copyright Takamitsu Endo (ryukau@gmail.com)
// SPDX-License-Identifier: Apache-2.0

export class Audio {
  cachedBuffer = null;
  constructor() {
    // Initialise AudioContext
    this.audioContext = new AudioContext();
  }
  play() {
    // If no AudioBuffer
    if (!this.cachedBuffer) {
      // Create AudioBuffer object
      let buffer = this.audioContext.createBuffer(
        this.audioContext.sampleRate);
      for (let i = 0; i < this.wave.channels; ++i) {
        // Write to AudioBuffer
        buffer.copyToChannel(new Float32Array(
          this.wave.data[i]), i, 0);
      }
      this.cachedBuffer = buffer;
    }
    // And set AudioContext's AudioBuffer to our's
    this.source =
      this.audioContext.createBufferSource();
    this.source.buffer = cachedBuffer;

    // And play
    this.source.start();
  }
}
```

Fig. 2. Heavily redacted version of the custom Audio class used by ryukau[6] at `common/wave.js` which uses the WebAudio API

REFERENCES

[1] B. Smus, *Web audio API: advanced sound for games and interactive apps*. O'Reilly Media, Inc., 2013.